KLAi

Forecasting and a first end-to-end model: baseline, time-based split and Pipeline

How do we predict next month, and how do we pack a whole modelling job into one object we can trust?

AI and Data Science Professional | Session 19 of 24 | Module D: Machine learning
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Split a time series by date and explain why a random split leaks the future

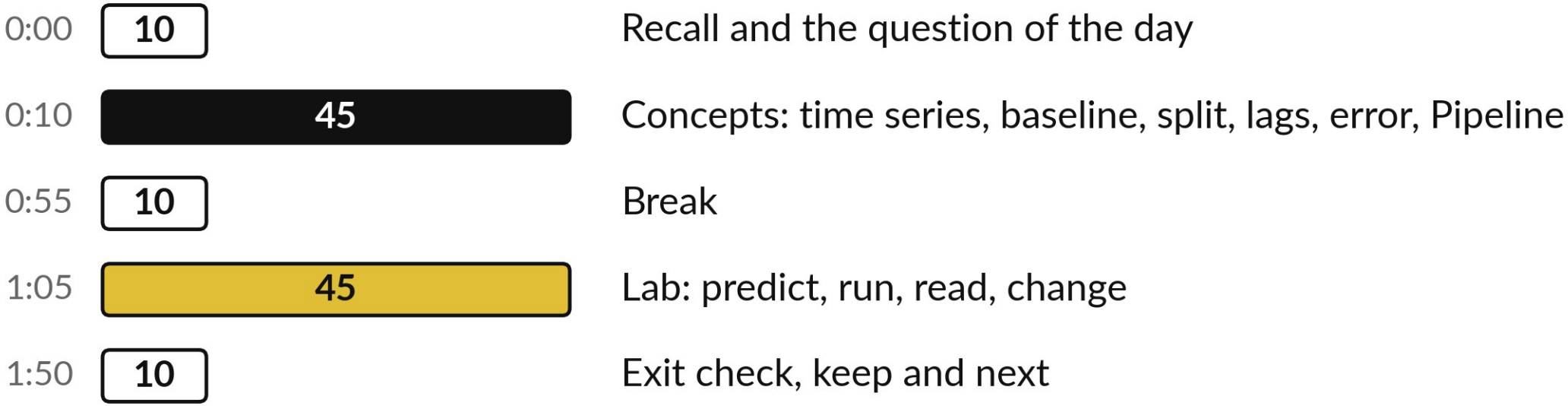


Beat a seasonal naive baseline with lag features and report the error in cases (MAE, RMSE, MAPE)



Build a scikit-learn Pipeline and write your Project milestone 3 result in three sentences

Your two hours: forecasting ideas first, then the Project milestone 3 lab



Ideas with pictures first. Then you open the notebook, forecast 2025 and write your result.

Three questions from last time

1. What is learning without a target variable called, and how did we judge the customer segments?

2. Why did we standardise the variables before running k-means?

3. What does the elbow method plot, and what do you look for?



Answer aloud first. Then we check together.

Answers to the three questions from Session 18

Question	Short answer
What is learning without a target variable called, and how did we judge the customer segments?	Unsupervised learning. With no right answer to check, we judged each cluster by whether it could be described in business words and acted on.
Why did we standardise the variables before running k-means?	k-means works with distances. A variable in large units, such as spend in UGX, would swamp a variable in small units, such as visits.
What does the elbow method plot, and what do you look for?	The inertia (within-cluster sum of squares) for each k. You look for the bend where another cluster stops reducing it much.

Words we use today

	Word	What it means
	**Time series (trend, seasonality, noise)**	One variable recorded at regular times, such as cases each month. It splits into a slow trend, a yearly seasonality and leftover noise.
	**Baseline model (seasonal naive forecast)**	The simplest forecast: copy the same month from last year. A model must beat it to be worth using.
	**Time-based split (temporal split)**	Train on the months before a cut date and test on the months after it. Never split a time series at random.
	**Lag feature**	An earlier value placed in the same row, such as last month's cases. It is built inside each district.
	**Pipeline (scikit-learn)**	Preprocessing steps and a model packed into one object, so training and test records go through the same steps.

A time series is one variable recorded at regular times: here 60 months for 12 districts

	A	B	C
	month	rainfall_mm	malaria_cases
1	2021-01	52.6	3794
2	2021-02	11.5	3291
3	2021-03	80.8	2871
4	2021-04	31.5	3870

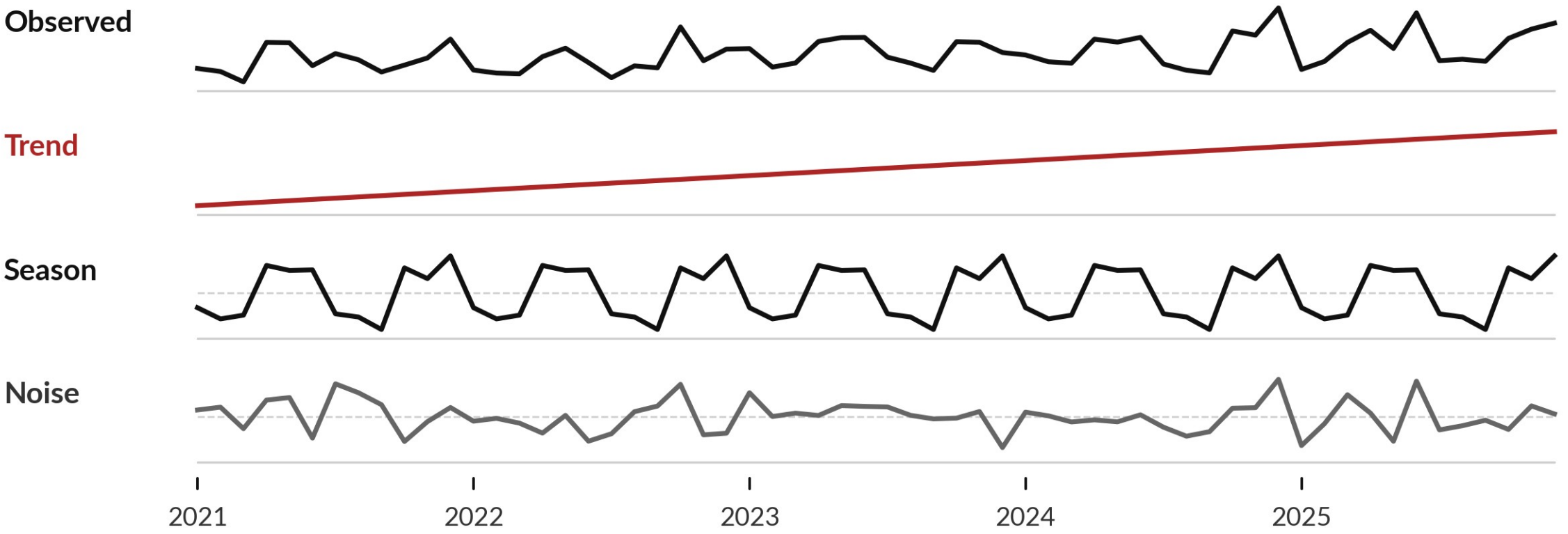
A time series is a variable recorded at regular times, in time order.

Each record is one district in one month: 720 records from clinic_visits.csv.

The picture shows the first months of Gulu, from 2021-01.

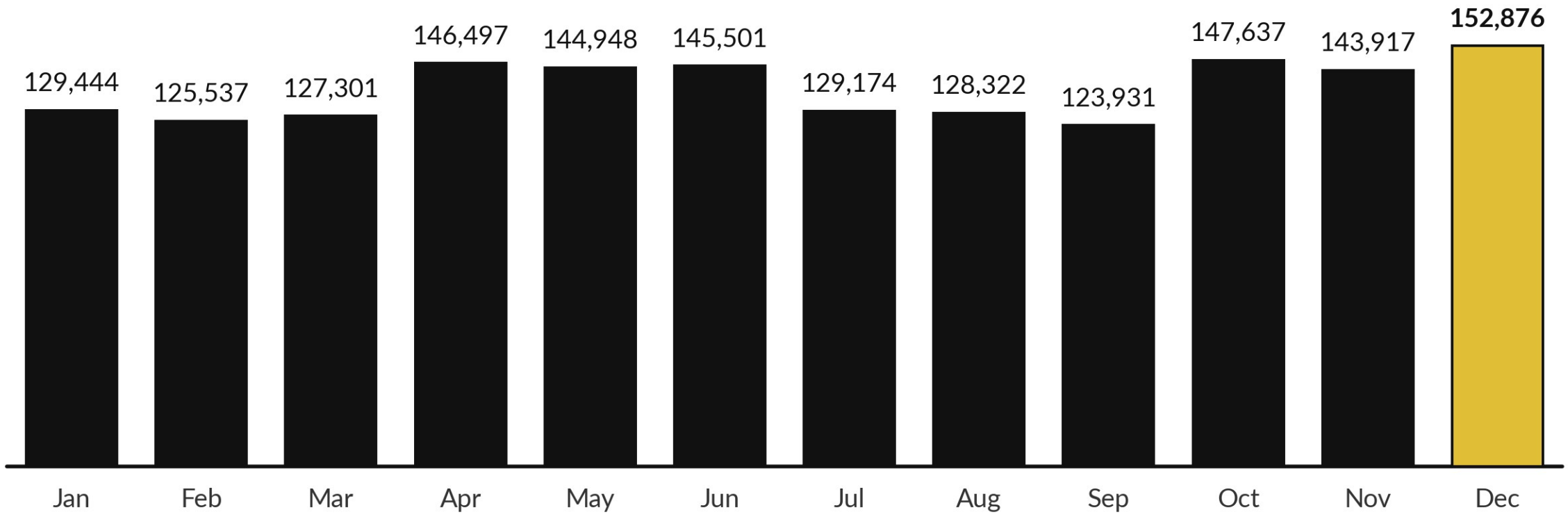
Forecasting means predicting the next values of the series.

A time series splits into trend, seasonality and noise (decomposition)



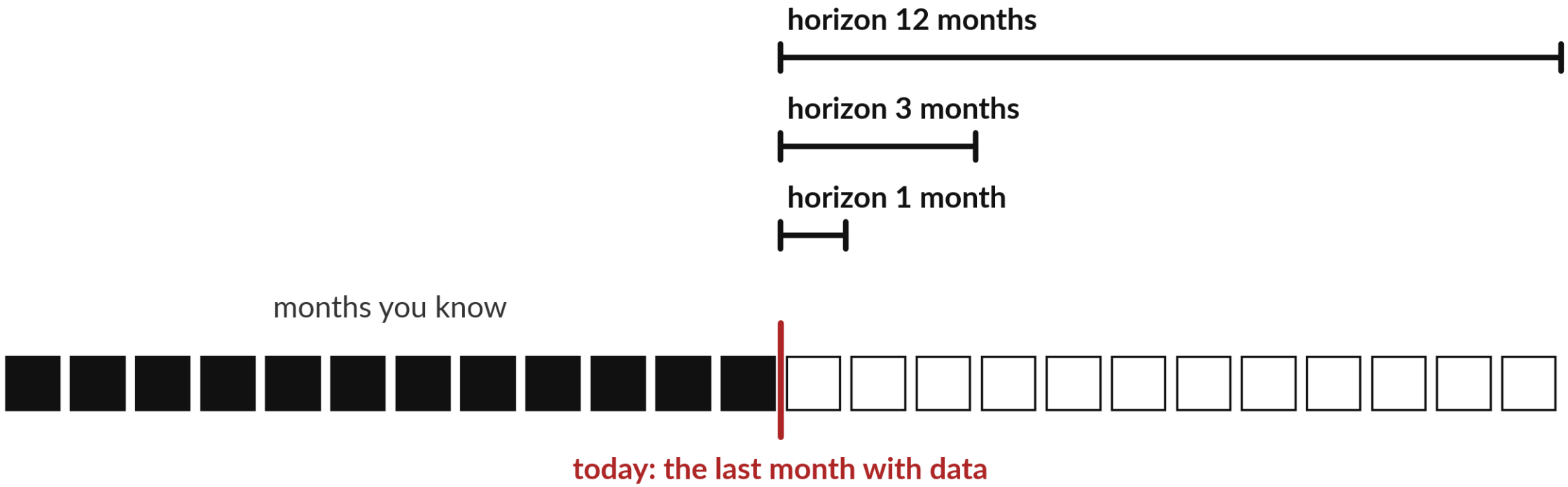
All 12 districts added up. Trend: about 69 more cases a month. Noise: the rest.

Seasonality repeats every year: December is the peak and September is the trough



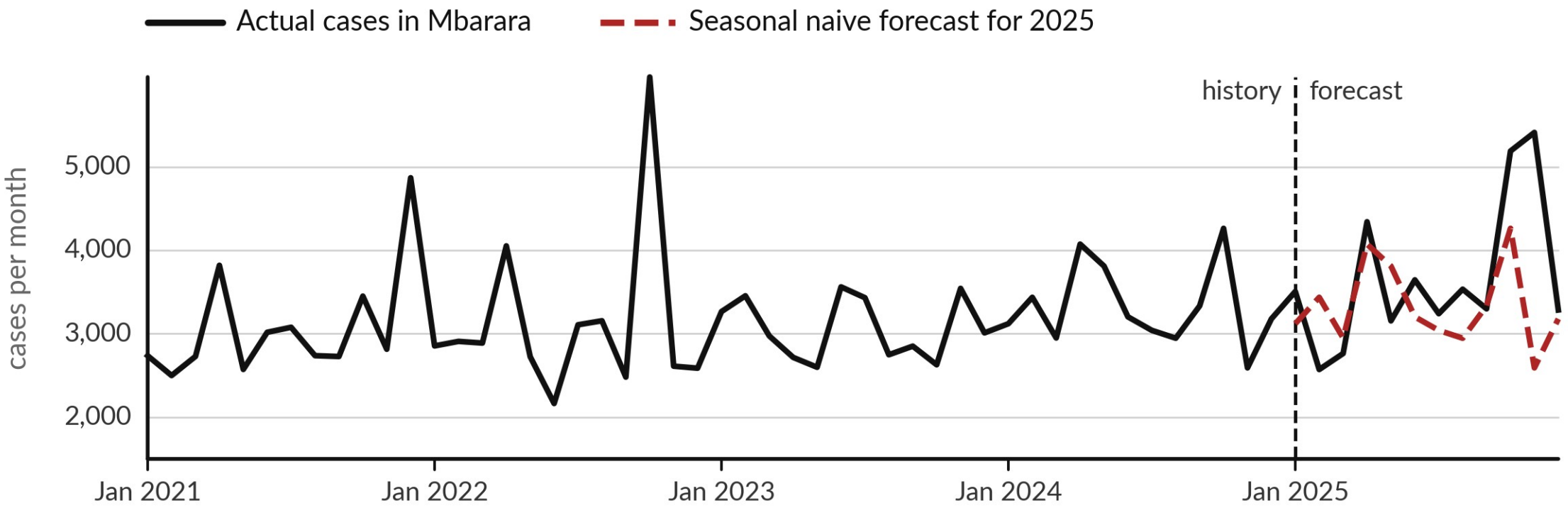
Cases of all 12 districts over five years, added for each calendar month. Gold: the peak.

The forecast horizon is how far ahead you predict



Horizon: months between the last month you know and the month you predict. Today: horizon 1.

The baseline model is a seasonal naive forecast: copy the same month from last year



Forecast for January 2025 = January 2024. Mbarara 2025: average miss 623 cases.

Mean absolute error (MAE) is the average size of the errors, in cases

Definition

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\, y_i - \hat{y}_i \,|$$

Mbarara 2025, n = 12

$$\text{MAE} = \frac{7{,}479}{12} = 623.25$$

Symbol	What it means	In our data
n	How many forecasts are scored.	12 months of 2025
y_i	The actual value for month i.	Mbarara, January 2025: 3,506 cases
ŷ_i (y-hat)	The forecast for month i.	Mbarara, January 2025: 3,120 cases
| |	Absolute value: drop the sign.	January miss: 386

MAE is in the unit of the target: cases. The sum of the 12 absolute errors is 7,479.

MAE, RMSE and MAPE are three ways to score the same forecast

	Measure	How it is computed	Unit	Baseline	Lag model
	**MAE**	Average of |actual minus forecast|	cases	460.7	338.0
	**RMSE (root mean squared error)**	Square each miss, average, take the square root. Big misses count more.	cases	748.6	490.9
	**MAPE (mean absolute percentage error)**	Average of |miss| divided by actual, times 100	percent	18.5	16.3

Scores on the 144 district-months of 2025. MAPE cannot be used when an actual value can be 0.

A time-based split trains on the past and tests on the future

Random split: wrong for a time series



Some training months come after the test months, so the model has seen the future.

Time-based split (temporal split): cut at 1 January 2025



Train on the 432 records of 2022 to 2024. Test on the 144 records of 2025.

Next month always lies after everything you know, so the test must too.

Data leakage from the future flatters a score

Looks ahead (leaks)

A random split moved MAE only a little here (332.2 against 338.0), but trend raises the risk.

A shift(-1) feature holds next month: MAE falls from 304.8 to 265.5.

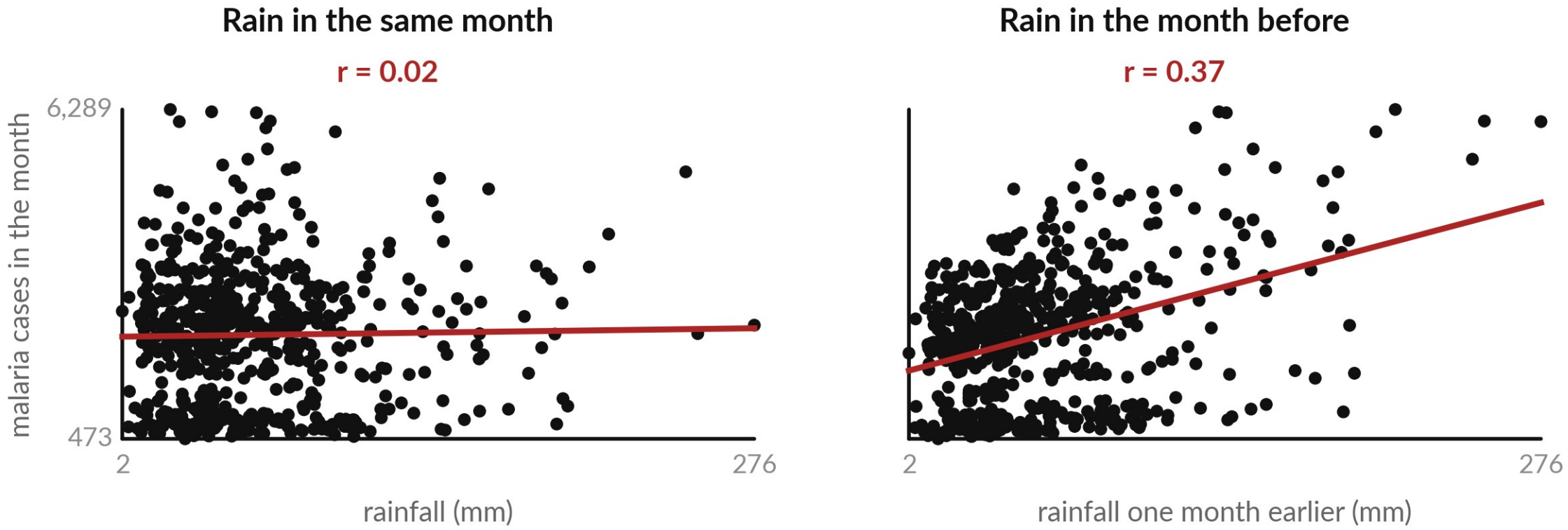
Safe

Cut at a date: train before, test after.

Use the past only: shift(1) or more.

A feature absent on forecast day must go.

Last month's rain relates to malaria cases (r=0.37) but this month's rain does not (r=0.02)



Each dot is one district-month (576 in all). r is the Pearson correlation.

A lag feature puts an earlier value into the same row

	A	B	C	D
	month	malaria_cases	cases_lag1	cases_lag12
1	2021-01	3794		
2	2021-12	3422	2985	
3	2022-01	3081	3422	3794
4	2022-02	3439	3081	3291

A lag feature is the value of a variable some months earlier, in the same record.

cases_lag1 is last month. cases_lag12 is the same month last year.

Gulu, 2022-01: the lag 12 value 3794 is the cases of 2021-01.

The first rows have no lag, so they are blank.

A lag built without groupby borrows values from the district above it

	district	month	malaria_cases	cases_lag1
2	Fort Portal	2025-12	3074	
3	Gulu	2021-01	3794	3074
4	Gulu	2021-02	3291	3794

Row 3 Shifting the whole column gives Gulu a lag of 3074: the December 2025 cases of Fort Portal. Gulu has no earlier month, so it should be blank.

Without groupby: 11 wrong lag 1 and 132 wrong lag 12 values. Test MAE 366.3.

Lag features cost records: 720 become 576, then 432 to train and 144 to test

	Step	Records	Why
▦	**All records in clinic_visits.csv**	720	12 districts times 60 months
✖	**Records with every lag feature**	576	The first 12 months of each district have no lag 12: 144 lost
📅	**Training set: January 2022 to December 2024**	432	12 districts times 36 months
⚑	**Test set: January 2025 to December 2025**	144	12 districts times 12 months

Lost rows are an expected cost. A lag needs history.

Read the recipe: lag features inside each district, then a split at the cut date

```
clinic = clinic.sort_values(['district', 'month'])
g = clinic.groupby('district')
clinic['cases_lag1'] = g.malaria_cases.shift(1)
clinic['cases_lag12'] = g.malaria_cases.shift(12)
clinic['rain_lag1'] = g.rainfall_mm.shift(1)
features = ['cases_lag1', 'cases_lag12', 'rain_lag1']
ready = clinic.dropna(subset=features)
train = ready[ready.month < '2025-01-01']
test = ready[ready.month >= '2025-01-01']
print(len(clinic), len(ready), len(train), len(test))
```

```
720 576 432 144
```

In plain words

Line 1 sorts by district and month. Line 2 groups by district, so every shift stays inside one district.

Lines 3 to 5 build the three lag features.

Line 7 drops the records that have no lag.

Lines 8 and 9 cut at 1 January 2025.

The four counts: 720, 576, 432 and 144.

Fit on the past, test on 2025: the lag model beats the baseline

```
X_train = np.c_[np.ones(len(train)), train[features]]
y_train = train.malaria_cases
coef = np.linalg.lstsq(X_train, y_train,
                       rcond=None)[0]
X_test = np.c_[np.ones(len(test)), test[features]]
y_pred = X_test @ coef
y_test = test.malaria_cases
baseline_mae = abs(y_test - test.cases_lag12).mean()
model_mae = abs(y_test - y_pred).mean()
print('baseline', round(baseline_mae, 1))
print('model   ', round(model_mae, 1))
```

```
baseline 460.7
model    338.0
```

In plain words

Lines 1 to 3 fit a linear regression with numpy, using only the training set.

The coefficients are the intercept and one weight for each lag feature.

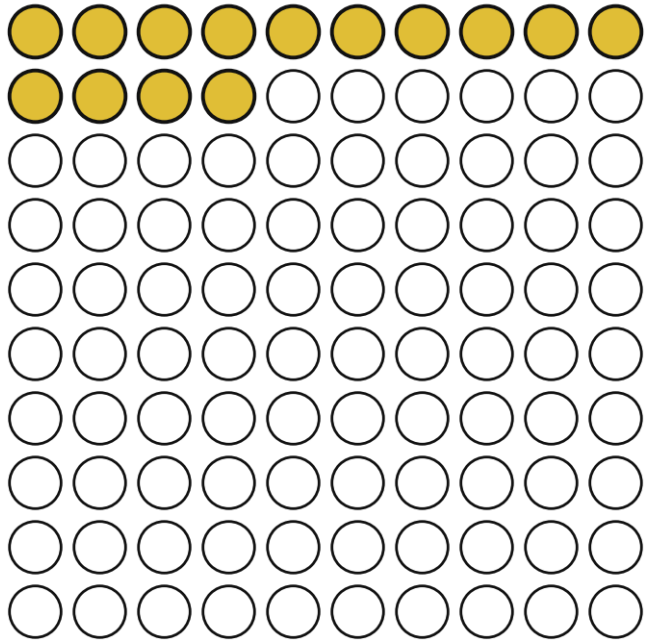
y_pred holds the forecast for each test record.

The baseline misses by 460.7 cases on average. The model misses by 338.0.

Judge the error in cases: an average miss of 338 is 14 percent of a typical month

14%

The lag model misses by 338.0 cases on average. The mean district-month in 2025 had 2,403 cases. The baseline misses by 19 percent of that.



percent of a typical month is the average miss

The model beats the baseline on average but not in every district

District	Typical month (cases)	Baseline MAE	Model MAE	Lower error
Gulu	3,730	980	725	Model
Mbarara	3,661	623	309	Model
Mukono	4,250	708	599	Model
Jinja	1,103	192	285	Baseline
Kabale	809	149	206	Baseline
Kampala	648	121	206	Baseline

The model has the lower error in 9 of 12 districts and in 83 of 144 district-months. Always look below the average.

The longer the horizon, the weaker the model: at 12 months it loses to the baseline

Horizon	Features used	Model MAE	Baseline MAE	Result
1 month	cases and rain h months back, and cases 12 months back	338.0	460.7	Model better
3 months	cases and rain h months back, and cases 12 months back	448.3	460.7	Model better
6 months	cases and rain h months back, and cases 12 months back	430.4	460.7	Model better
12 months	cases and rain 12 months back	480.1	460.7	Baseline better

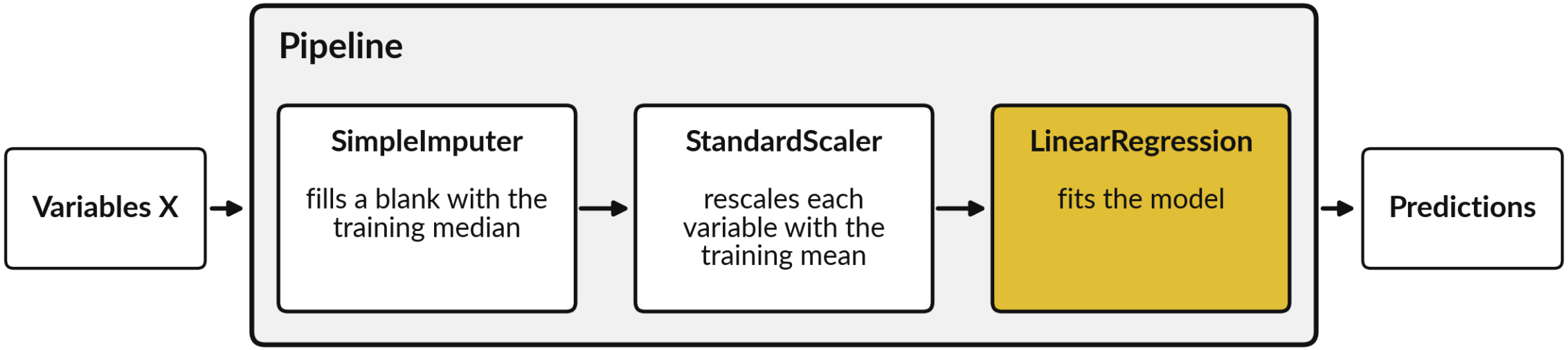
Each row refits the model with lags that are known h months ahead. Same train and test split.

Walk-forward validation repeats the cut date, so one lucky year cannot fool you

	Cut date	Train records	Test year	Baseline MAE	Model MAE
📅	**1 January 2023**	144	2023	484.6	327.6
📅	**1 January 2024**	288	2024	469.3	336.8
📅	**1 January 2025**	432	2025	460.7	338.0

Walk-forward validation (expanding window). In scikit-learn the same idea is TimeSeriesSplit.

A Pipeline packs preprocessing and the model into one object



fit(X_train, y_train): every step learns only from the training records.

predict(X_test): the same steps run on the test records. Nothing is re-learned.

One object to fit, one to predict, one to save. It also stops test records leaking into the preprocessing.

Read the recipe: a scikit-learn Pipeline gives the same forecast in a few lines

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
pipe = make_pipeline(SimpleImputer(strategy='median'),
                     StandardScaler(),
                     LinearRegression())
pipe.fit(train[features], train.malaria_cases)
y_pipe = pipe.predict(test[features])
print(round(abs(y_test - y_pipe).mean(), 1))
```

```
338.0
```

In plain words

make_pipeline chains the three steps in order.

fit learns the median, the mean and the weights from the training records only.

predict runs the same steps on the test records.

The MAE is 338.0, the same as the numpy fit: standardising does not change a linear regression forecast.

Adding the district as indicator variables lowers the MAE from 338 to 217

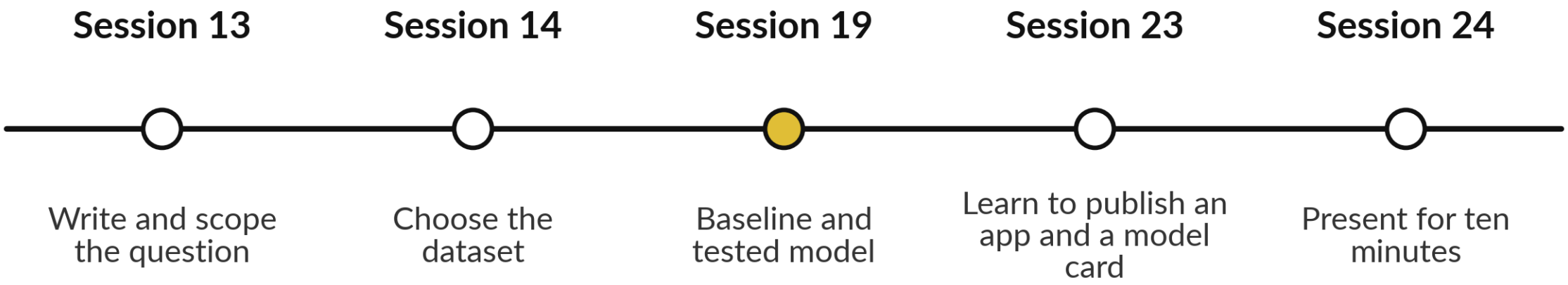
	Model	Variables	MAE (cases)
	**Pipeline on three lag features**	3	338.0
	**Same Pipeline with 11 district indicator variables**	14	217.1

One-hot encoding: district becomes 0/1 indicator (dummy) variables. Drop one district to avoid the dummy variable trap. In scikit-learn: ColumnTransformer and OneHotEncoder.

Where you will use this at work: forecast next month, then say how wrong you may be

	Your job	The series you meet	What today lets you do
	**Records officer at a district health office, using Excel and pandas**	Malaria or clinic cases per district per month	Forecast next month and tell the pharmacy how many extra cases to stock for.
	**Sales analyst at a retail chain**	Weekly orders per branch (Lakeside Mart style)	Forecast next week with a baseline and a lag model, tested by date.
	**Credit analyst at a microfinance institution**	Monthly collections or repayments	Check that no feature looks ahead before you report a score.
	**Analyst at a mobile money agent network**	Daily or weekly transactions (illustrative)	Package preparation and model in a Pipeline you can rerun every month.

Course project timeline: today is the baseline and tested model

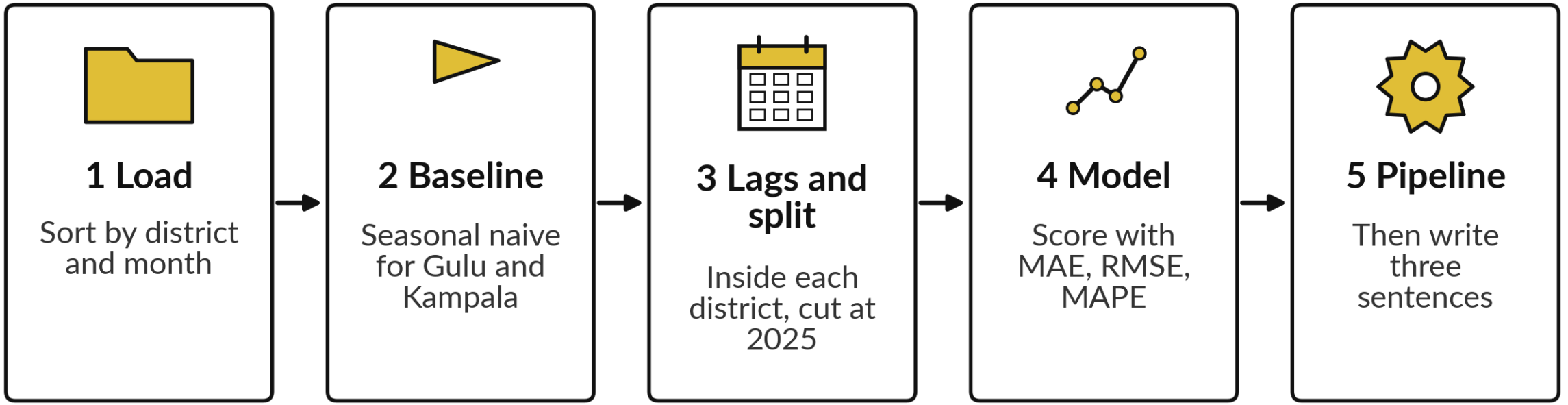


Milestone 3: baseline, unseen-record score, error in business units.

Part 2: the forecasting lab

Close the slides. Open the notebook.

The lab has five parts and ends with your Project milestone 3 results file



Predict, run, read, change one thing. Automatic checks tell you when a number is right. Keep the results file the notebook writes.

Check your numbers before you move on

What you compute	You should get
Records in the file, and blank total_visits	720 and 18
Gulu baseline MAE, and Kampala baseline MAE in 2025	980.2 and 120.8
Correlation of cases with this month's rain and with last month's rain	0.02 and 0.37
Records with every lag, training records and test records	576, 432 and 144
Baseline MAE, model MAE	460.7 and 338.0
Model RMSE and model MAPE	490.9 and 16.3 percent
MAE with the district indicator variables	217.1
MAE when the lag is built without groupby	366.3

If your number differs, find the task that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

What you see	What it means	What to do
Test records are not 144	The cut date or the dropna step differs.	Use month >= '2025-01-01' and drop records missing a lag first.
The model scores worse than the baseline	The features or the lag direction are wrong.	Check that you used shift(1). A shift(-1) looks ahead. Check the sort order.
NaN in the predictions	A lag feature is blank in a test record.	Drop the blank records with dropna(subset=features) before you split.
KeyError: 'cases_lag12'	The lag column was not created, or it has another name.	Run the lag cell again, and check the spelling.
Your MAE differs from the table by a little	A different cut date, features or dropped row.	Below 0.1 case is rounding. Larger means other features.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**A seasonal naive forecast for March 2026 uses which number?**	The March 2025 value: the same month last year.
	**Why must a forecast be tested on later months?**	Next month always lies after the data you have. A random split lets the model see the future, which is data leakage.
	**Your model misses by 338 cases and the baseline by 461. What do you tell the manager?**	It beats the baseline by about 27 percent, with an average miss near 338 cases at horizon 1, but some districts and months miss by more.

Save your notebook: it is your Project milestone 3

What you keep from today

Keep your Project milestone 3 notebook: a baseline, a model tested on 2025, the error in cases with MAE, RMSE and MAPE, a Pipeline, and three sentences for a district manager.

Our question today

How do we predict next month, and how do we pack a whole modelling job into one object we can trust?

Next session

Session 20: Neural networks and deep learning. One neuron and a training loop in numpy, then a network against a forest.

Take away

Your lab file

A forecast notebook

The key test

Model beats the baseline

Next session

Neural networks